Protein Modeling with AlphaFold

Proteins, nucleic acids and ligands - in one model

A 60-minute hands-on workshop

Northwestern IT · Research Computing and Data Services

Northwestern

BEFORE WE START

Setup…

1. **Slides and scripts are here**

```
github.com/nuitrcs/Protein-Modeling
```

2. **Log on to Quest, via terminal or Quest OnDemand**

```
ssh <netid>@login.quest.northwestern.edu
```

3. **Move to our classroom folder**

```
cd /projects/e32559
```

4. **Make your own subfolder and go into it**

```
mkdir <folder_name> && cd <folder_name>
```

5. **Copy today's folder into it**

```
cp -R /projects/e32559/06_alphafold/ .
```

BENCHMARKS

What AlphaFold 3 does and how well

AlphaFold, from Google DeepMind, predicts a protein's 3D structure from its amino acid sequence with high accuracy. AlphaFold 3 extends this to complexes containing non-protein molecules: DNA, RNA, ligands and ions, and is available for non-commercial use through AlphaFold 3 GitHub repository.

Protein chains

with post-translational modifications

DNA and RNA

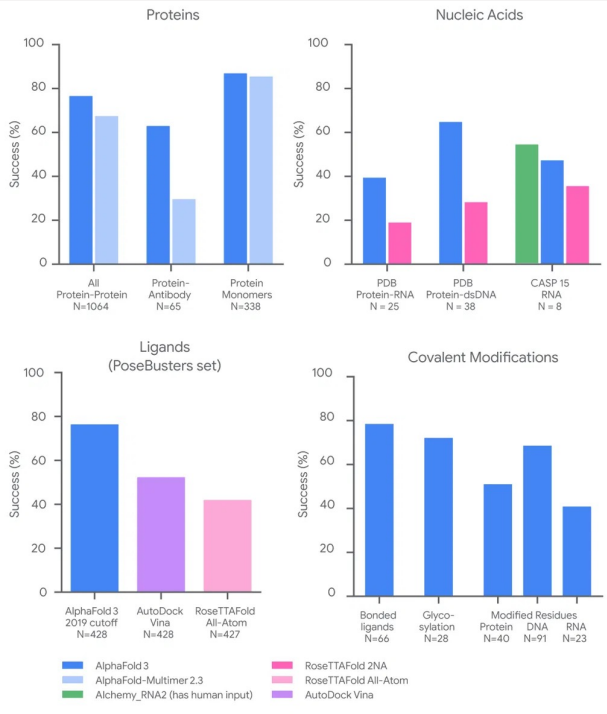
including modified bases

Ligands and ions

by CCD code or SMILES

Covalent bonds

glycans, covalent inhibitors



Antibodies are the big win
Roughly double the previous rate. Much of that comes from AF3 depending less on the MSA, which is exactly what antibody loops lack.

It beats a docking program at docking
On PoseBusters, without being given a pocket. But see the limits slide before you build a screen on this.

RNA is the exception
AF3 leads automated methods but does not beat systems with human expert intervention in the loop.

Comparators: AlphaFold-Multimer 2.3 for the protein rows, AutoDock Vina for ligands. Approximate values read from Abramson et al. 2024, Fig. 1.

IF YOU HAVE USED AF2

What changed from AlphaFold 2

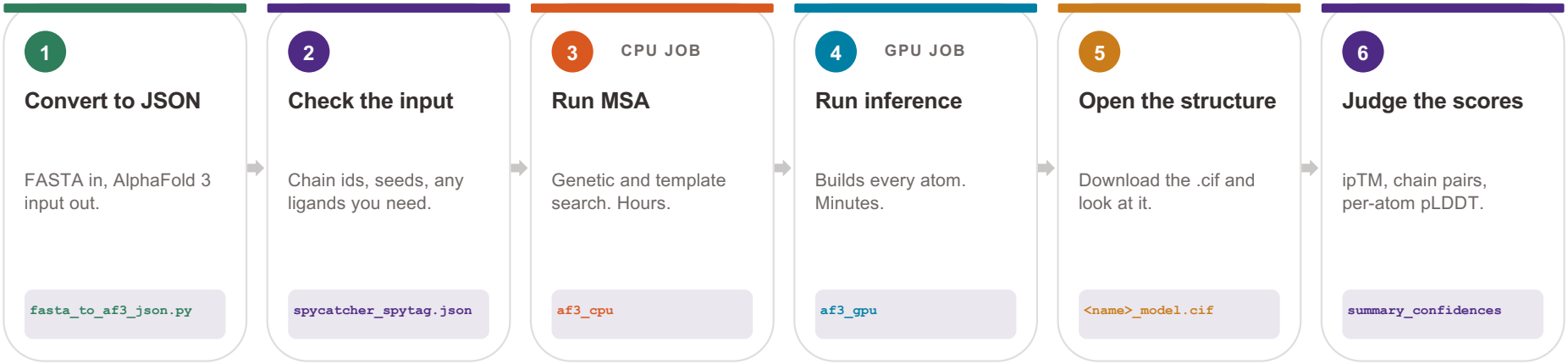
	AlphaFold 2	AlphaFold 3
Scope	protein monomers and multimers	any complex: protein, DNA, RNA, ligand, ion
Input format	FASTA file	JSON file - the biggest practical change
Structure output	PDB (ranked_0.pdb …)	mmCIF only (<job>_model.cif)
pLDDT	per residue, in the B-factor column	per atom, in confidences.json
Relaxation	Amber minimisation step	none - diffusion emits all atoms directly
Ranking	mean pLDDT, or pTM + ipTM	ranking_score in ranking_scores.csv
GPU on Quest	A100 only	A100 or H100

AlphaFold 2 is not obsolete.
AF2 is Apache-2 licensed and usable commercially; AF3 is non-commercial only, with restrictions on what you may do with the outputs. AF2 is also the better fit for screening thousands of interactions. Read the terms attached to your parameters before you build a pipeline on AF3.

THE WHOLE THING, END TO END

Six steps, two jobs

Everything in this session is one of these six steps. Each one hands a specific file to the next and it starts with turning a FASTA into JSON.



Steps 3 and 4 are separate jobs

The search needs no GPU and takes hours; inference needs a GPU and takes minutes. Running them as one job holds an A100 idle all afternoon.

THE KEY IDEA

Why it needs a multiple sequence alignment (MSA)

You supply a sequence. What the network reads is a multiple sequence alignment (MSA) - your sequence set against every relative it can find in the database.

FIVE HOMOLOGS, ONE REGION

V	K	T	**L**	T	G	K	**T**
V	K	S	**L**	T	G	K	**T**
V	K	T	**M**	T	G	K	**S**
V	K	S	**M**	T	G	K	**S**
V	K	T	**L**	T	G	K	**T**

In the MSA, column 4 changes and column 8 changes with it. Those two residues are probably touching.

Evolve independently → probably not in contact.

1 **Covariation is the signal**

Two residues in contact constrain each other: a destabilising mutation in one tends to be compensated in the other. Columns that change together are the evidence.

2 **Every pair gets a representation**

The network builds a pair representation for every pair of residues, however distant, encoding the relationship the MSA implies. That is what becomes distances and relative positions.

3 **Information flows both ways**

The MSA and the pair map keep updating each other, refining the structural hypothesis before any coordinates exist. Templates help only when the MSA is thin.

AlphaFold 3 leans on the MSA less which makes it more versatile

AF3 reads the MSA more lightly. The part of the network that handles the MSA is smaller and simpler than AlphaFold 2's.

That is why antibody work improved. The binding loops of an antibody change constantly from one antibody to the next, so there is little shared pattern in the MSA to find. Relying on it less helps exactly there.

But the MSA still does most of the work. For proteins, AF3 still leans heavily on which columns change together.

Learn more about how alphafold3 model architecture here.

PART TWO

Getting it to run

FIRST, THE WEIGHTS

Getting the model weights

The code is on Quest already. The weights are a separate ~1 GB file, and they come with conditions.

1 **Read the terms of use**

```
github.com/google-deepmind/alphafold3 → WEIGHTS_TERMS_OF_USE.md
```

Non-commercial use only, no redistribution of the weights, and restrictions on what you may do with the outputs. Read it before you download, not after.

2 **Download the weights**

```
wget https://storage.googleapis.com/alphafold3/af3.bin.zst
```

A straight download - no request form, no waiting. Put it somewhere you own that a compute node can read.

3 **Point every run at that folder**

```
--model_dir=/path/to/your/af3/models
```

af3_gpu takes this flag. The GPU job will not start without it.

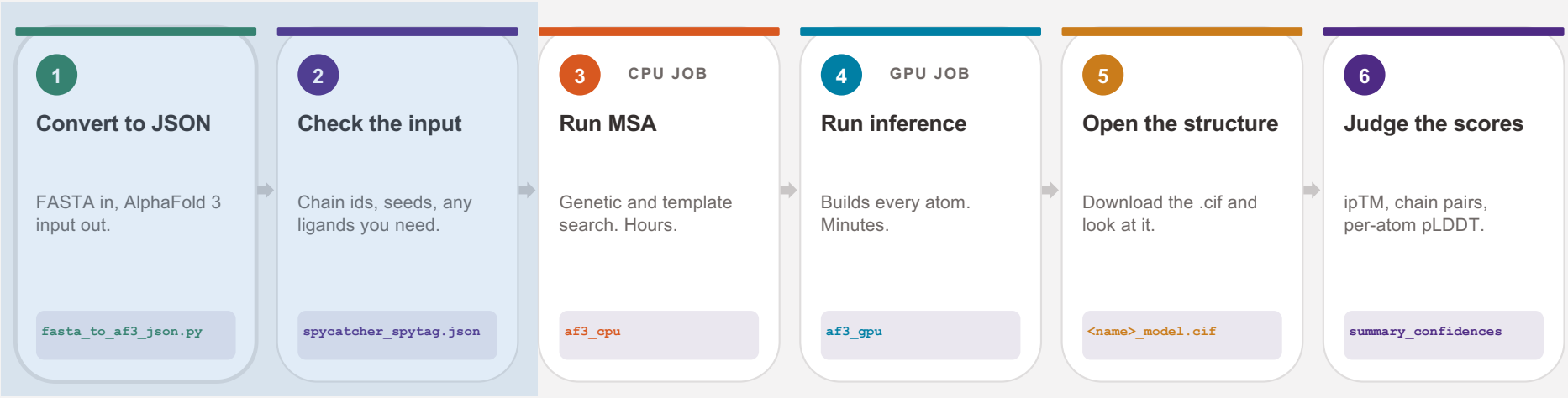
For today, none of this is in your way - the parameters are already downloaded for the workshop.

You can run this hour without doing steps 1 and 2. Do them properly before you use AlphaFold 3 for your own work.

THE WHOLE THING, END TO END

Six steps, two jobs

Everything in this session is one of these six steps. Each one hands a specific file to the next and it starts with turning a FASTA into JSON.



STEP ONE

From FASTA to AlphaFold 3 JSON

You have a FASTA already. This script turns one into a valid AlphaFold 3 input file.

Open fasta to have a look:

```
cat spycatcher_spytag.fasta
```

FASTA in

```
>6M0J_1|Chains A|ACE2|Homo sapiens
STIEEQAKTFLDKFNHEAEDLFYQSS...
>6M0J_2|Chains E|Spike|SARS-CoV-2
RVQPTESIVRFPNITNLCPFGEVFNA...
```

JSON out

```
{
  "name": "6M0J",
  "sequences": [
    {"protein": {"id": ["A"], "sequence": "STIE..."}},
    {"protein": {"id": ["E"], "sequence": "RVQP..."}}
  ],
  "modelSeeds": [1], "dialect": "alphafold3",
  "version": 1
}
```

```
# one file, or many at once
module load python/3.12.10
python fasta_to_af3_json.py spycatcher_spytag.fasta
```

Writes spycatcher_spytag.json beside the FASTA.

What it works out for you

Chain ids from RCSB headers ("Chains A, C"), or A, B, C in order. Protein, DNA or RNA from the letters in the sequence. Name from the filename, seed set to 1.

It gets you a valid file — not necessarily the right one.

Open it and check before you submit: are the chain ids what you expect, is the copy number right, and does anything need adding by hand — a ligand, an ion, a modification, extra seeds? The script only knows what was in the FASTA.

STEP TWO

The input file with the amino acid sequence is a JSON

Open json to have a look:

```
cat spycatcher_spytag.json
```

spycatcher_spytag.json: SpyCatcher (chain A) and its SpyTag peptide (chain B)

```
{
  "name": "spycatcher_spytag",
  "sequences": [
    {"protein": {
        "id": ["A"],
        "sequence": "VDTLSGLSSEQ..."
    }},
    {"protein": {
        "id": ["B"],
        "sequence": "AHIVMVDAYKPTK"
    }}
  ],
  "modelSeeds": [1],
  "dialect": "alphafold3", "version": 1
}
```

name
Sanitised and used for the output directory and every filename in it.

id
One letter per chain. A list means several identical copies — that is how you set stoichiometry.

modelSeeds
One structure per seed. At least one is required; AF3 will not pick for you.

dialect / version
Must be "alphafold3". The server format is different and gets converted.

STEP TWO

Adding a ligand, DNA, or RNA

Everything beyond protein goes in the same sequences list. Refer to https://github.com/google-deepmind/alphafold3/blob/main/docs/input.md for more guidance.

```
"sequences": [
  {"protein": {"id": "A", "sequence": "..."}},

  {"dna": {"id": "C", "sequence": "GACCTCT"}},
  {"rna": {"id": "E", "sequence": "AGCU"}},

  {"ligand": {"id": "F",
              "ccdCodes": ["ATP"]}},
  {"ligand": {"id": "G",
              "smiles": "CC(=O)OC1C..."}},
  {"ligand": {"id": "H",
              "ccdCodes": ["MG"]}}
],

"bondedAtomPairs": [
  [["A", 145, "SG"], ["L", 1, "C04"]]
]
```

Ligands: two ways
A CCD code if the molecule is in the Chemical Component Dictionary; a SMILES string if it is not.

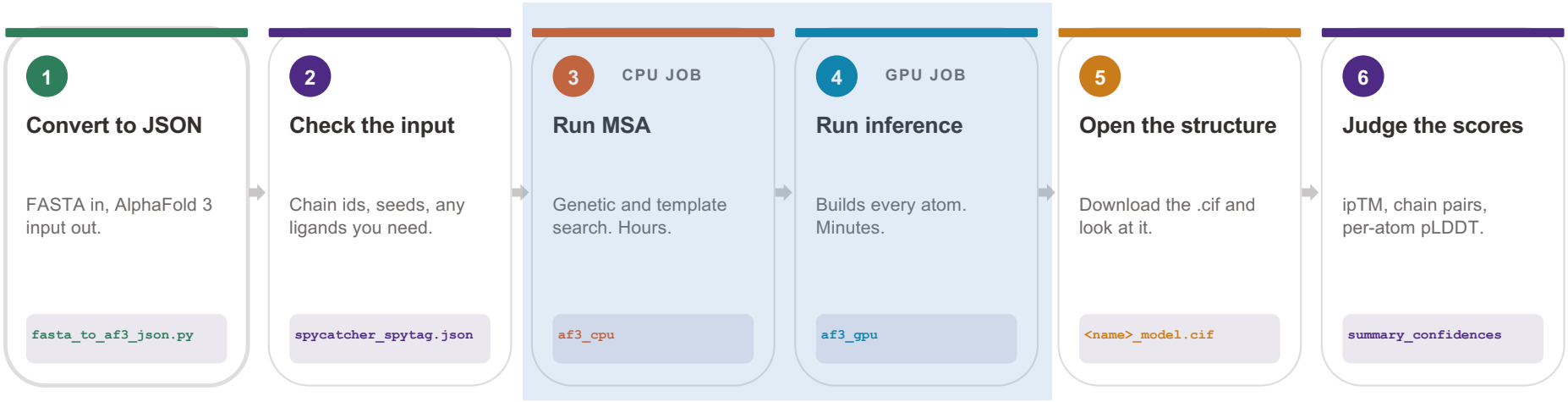
Ions are ligands
A magnesium ion is just a ligand with ccdCodes ["MG"]. There is no separate ion type.

Modifications inline
Phosphorylation, methylated bases and the like go in a modifications list with a CCD code and a position.

Covalent bonds
Three parts per atom: chain id, residue number, atom name. Here chain A's Cys145 sulfur (SG) bonds to atom C04 of ligand L — a covalent inhibitor.

THE WHOLE THING, END TO END

Six steps, two jobs



Steps 3 and 4 are separate jobs

The search needs no GPU and takes hours; inference needs a GPU and takes minutes. Running them as one job holds an A100 idle all afternoon.

QUEST MECHANICS

The two job scripts and two json inputs

One CPU job to build the MSA, one GPU job to predict the structure. Submit the first, wait for it, then submit the second.

STAGE 1 `msa_generation_cpu.sh` — CPU only, hours

```
#!/bin/bash
#SBATCH --account=e32559
#SBATCH --partition=normal
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=8
#SBATCH --time=3:00:00
#SBATCH --mem=6G
#SBATCH --output=logs/af3_cpu_%j.log

module purge
module load alphafold/3.0.0
mkdir -p logs

af3_cpu \
  --model_dir=$(pwd)/model \
  --output_dir=$(pwd)/msa_outputs \
  --json_path=$(pwd)/spycatcher_spytag.json
```

STAGE 2 `running_alphafold.sh` — one A100, minutes

```
#!/bin/bash
#SBATCH --account=e32559
#SBATCH --partition=gengpu
#SBATCH --gres=gpu:a100:1
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --time=00:10:30
#SBATCH --mem=8G
#SBATCH --output=logs/af3_%j.log

module purge all
module load alphafold/3.0.0
mkdir -p logs output_pdbs

af3_gpu \
  --model_dir=$(pwd)/model \
  --output_dir=$(pwd)/output_pdbs/ \
  --json_path=$(pwd)/msa_outputs/spycatcher_spytag/spycatcher_spytag_data.json
```

1. **Submit them in order** — sbatch msa_generation_cpu.sh , wait for it to finish, then running_alphafold.sh. Stage 2 needs stage 1's output file.
2. **$(pwd) means "where I am"** — Both scripts use it, so they work in whatever folder you copied them into - nothing to edit but the account line.
3. **Watch the handoff** — Stage 2 reads msa_outputs/<name>/<name>_data.json - the pipeline's output, not the original input file.

QUEST MECHANICS

The two job scripts and two json inputs

One CPU job to build the MSA, one GPU job to predict the structure. Submit the first, wait for it, then submit the second.

STAGE 1 msa_generation_cpu.sh — CPU only, hours

```
#!/bin/bash
#SBATCH --account=p12345
#SBATCH --partition=normal
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=8
#SBATCH --time=2:00:00
#SBATCH --mem=32G
#SBATCH --output=logs/af3_cpu_%j.log

module purge
module load alphafold/3.0.0
mkdir -p logs

af3_cpu \
  --model_dir=$(pwd)/model \
  --output_dir=$(pwd)/msa_outputs \
  --json_path=$(pwd)/spycatcher_spytag.json
```

STAGE 2 running_alphafold.sh — one A100, minutes

```
#!/bin/bash
#SBATCH --account=p12345
#SBATCH --partition=gengpu
#SBATCH --gres=gpu:a100:1
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --time=3:00:00
#SBATCH --mem=8G
#SBATCH --output=logs/af3_%j.log

module purge all
module load alphafold/3.0.0
mkdir -p logs output_pdbs

af3_gpu \
  --model_dir=$(pwd)/model \
  --output_dir=$(pwd)/msa_outputs \
  --json_path=$(pwd)/msa_outputs/spycatcher_spytag/spycatcher_spytag_data.json
```

- cat msa_generation_cpu.sh — to view the cpu job
- sbatch msa_generation_cpu.sh - to submit the cpu job
- cat running_alphafold.sh — to view the gpu job
- sbatch running_alphafold.sh - to submit the gpu job

1. **Submit them in order** — sbatch run_msa.sh, wait for it to finish, then sbatch run_af3.sh. Stage 2 needs stage 1's output file.
2. **$(pwd) means "where I am"** — Both scripts use it, so they work in whatever folder you copied them into - nothing to edit but the account line.
3. **Watch the handoff** — Stage 2 reads msa_outputs/<name>/<name>_data.json - the pipeline's output, not the original input file.

YOUR TURN

Submit a real prediction

SpyCatcher plus SpyTag. Inference only - we ran the data pipeline last week and staged the MSAs.

```
vim running_alphafold.sh #if you would like to edit the job
# i to start editing
# set --model_dir to your parameters
# set --output_dir to your folder
# Esc, then :wq and Enter to save and exit

sbatch running_alphafold.sh
```

While it queues

- No parameters yet? Pair up with a neighbour who has them
- Queue waits are normal - you have not broken anything
- We copy rather than read in place because AF3 writes into the output directory

Raise a hand the moment something looks wrong. Everyone gets to the outputs section either way.

WHILE IT RUNS

Check on your job

`squeue -u <netid>`	Everything running or pending right now. PD means pending, R means running.
`sacct -X`	Every job you submitted today, including the ones that already finished or failed.
`checkjob <jobid>`	Quest-specific: the details of one job, including why it is still waiting.
`tail -f logs/af3-<id>.log`	The live output. AF3 prints each sample as it finishes.

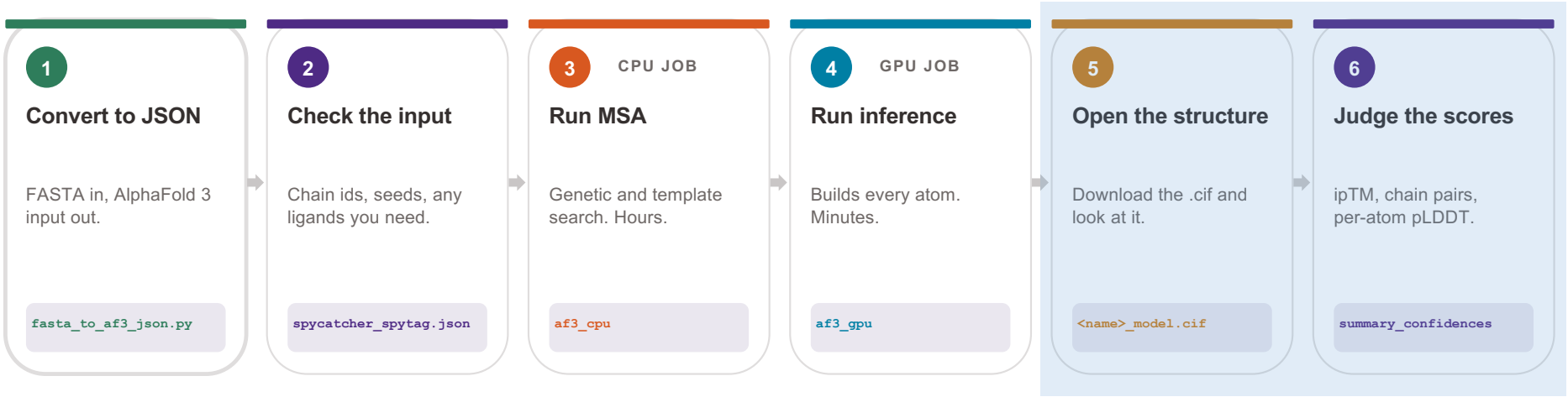
PART THREE

Reading what came back

`output_pdbs`

THE WHOLE THING, END TO END

Six steps, two jobs



Steps 3 and 4 are separate jobs

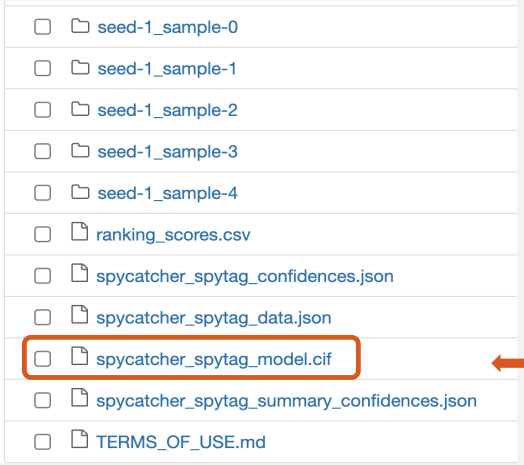
The search needs no GPU and takes hours; inference needs a GPU and takes minutes. Running them as one job holds an A100 idle all afternoon.

THE OUTPUTS

What comes back

`cd output_pdbs/spycatcher_spytag/`

this is a real run of our SpyCatcher–SpyTag job on Quest, straight out of the output directory.



Start here: the top-ranked model

`spycatcher_spytag_model.cif`
The top-ranked sample, in mmCIF. No PDB file is written.

`..._summary_confidences.json`
ptm, iptm, ranking_score, has_clash, fraction_disordered, and the chain-pair arrays. 332 bytes - read it in a text editor.

`..._confidences.json`
The big arrays: atom_plddts, pae, contact_probs, token_chain_ids.

`ranking_scores.csv`
All five samples with their scores. Note: no job-name prefix on this one.

`..._data.json`
Your input plus the MSAs the pipeline found. The 9.4 MB is almost all MSA.

`seed-1_sample-0 … -4`
The five diffusion samples. The winner is copied up to the top level.

FIRST LOOK

Open it and color it by confidence

Before any numbers: look at the structure and let pLDDT tell you which parts to trust.

1 **Find it**

```
output_pdbs/spycatcher_spytag/
  spycatcher_spytag_model.cif
```

2 **Download it**

```
scp
<netid>@quest.northwestern.edu:/home/<netid>/e32559/<name>/output_pd
bs/spycatcher_spytag/spycatcher_spytag_model.cif ~/Downloads/
```

3 **Open it**

```
PyMOL, or drag into
  pymol viewer
```

mmCIF opens in both — no conversion to PDB needed.

Colour by pLDDT

The score rides in the B-factor field of the .cif, so any viewer can do it:

```
Copy text from plddt_pymol.txt in your pymolrc
PyMOL:  plddt
```

> 90	Very high — trust the side chains too
70 – 90	Confident — backbone is reliable
50 – 70	Low — treat with real caution
< 50	Very low — do not interpret

What you are looking for

Two chains in contact — the SpyTag peptide in the SpyCatcher groove, not floating off on its own.

pLDDT is per atom in AlphaFold 3

Ligands and nucleotides get scores too, on the same scale. AlphaFold 2 only ever scored amino acids.

FIRST LOOK

Open it and color it by confidence

Before any numbers: look at the structure and let pLDDT tell you which parts to trust.

1 Find it

```
output_pdbs/spycatcher_spytag/
  spycatcher_spytag_model.cif
```

2 Download it

```
scp
<netid>@quest.northwestern.edu:/home/<netid>/e32553/<name>/output_pd
bs/spycatcher_spytag/spycatcher_spytag_model.cif ~/Downloads/
```

3 Open it

```
PyMOL, or dr
  pymol viewer
```

mmCIF opens in both — no conversion to PDB needed.

Colour by pLDDT

The score rides in the B-factor field of the .cif, so any viewer can do it:

```
Copy text from plddt_pymol.txt in your plymolrc
PyMOL:  plddt
```

> 90	Very high — trust the side chains too
70 – 90	Confident — backbone is reliable
50 – 70	Low — treat with real caution
< 50	Very low — do not interpret

What you are looking for

Two chains in contact — the SpyTag peptide in the SpyCatcher groove, not floating off on its own.

pLDDT is per atom in AlphaFold 3

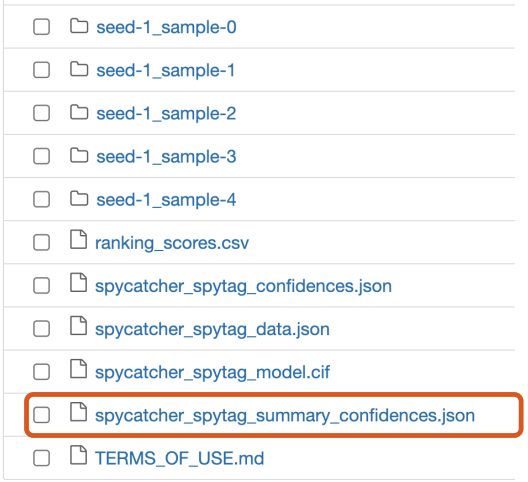
Ligands and nucleotides get scores too, on the same scale. AlphaFold 2 only ever scored amino acids.

You don’t have to do this part!
You can watch me open this structure on PyMOL

THE OUTPUTS

What comes back

This is a real run of our SpyCatcher–SpyTag job on Quest, straight out of the output directory.



Second look: confidence of best model

spycatcher_spytag_model.cif

The top-ranked sample, in mmCIF. No PDB file is written.

...summary_confidences.json

ptm, iptm, ranking_score, has_clash, fraction_disordered, and the chain-pair arrays. 332 bytes - read it in a text editor.

...confidences.json

The big arrays: atom_plddts, pae, contact_probs, token_chain_ids.

ranking_scores.csv

All five samples with their scores. Note: no job-name prefix on this one.

...data.json

Your input plus the MSAs the pipeline found. The 9.4 MB is almost all MSA.

seed-1_sample-0 … -4

The five diffusion samples. The winner is copied up to the top level.

READING CONFIDENCE

The scores, in one 332-byte file

cat spycatcher_spytag_summary_confidences.json.
These five numbers tell you whether to believe the structure you just looked at.

```
{
  "ptm": 0.71,
  "iptm": 0.83,
  "ranking_score": 0.79,
  "fraction_disordered": 0.12,
  "has_clash": false,
  "chain_ptm": [0.74, 0.03],
  "chain_iptm": [0.83, 0.83],
  "chain_pair_iptm": [[0.74, 0.83],
                      [0.83, 0.03]],
  "chain_pair_pae_min": [[0.6, 1.9],
                         [1.9, 0.7]]
}
```

Shape of the file, with illustrative numbers — chain A first, chain B second.

ranking_score *Sorts the five samples. Nothing else.*	0.8×ipTM + 0.2×pTM + 0.5×disorder − 100×clash, range −100 to 1.5. The clash and disorder terms make it useless as a quality score.
iptm *interface predicted TM-score - is the interface correctly predicted?*	Above 0.8 confident, below 0.6 sceptical, 0.6–0.8 grey. For a two-chain job this is the number to quote.
ptm *predicted TM-score — is the overall fold correctly predicted?*	Above 0.5 the fold may resemble the truth. Averaged over everything, so it hides good interfaces inside bad structures.
chain_pair_iptm *ipTM for one named pair of chains.*	Off-diagonal (i,j) is the ipTM for that one interface; the diagonal (i,i) is the pTM for chain i alone. This is where you look with three or more chains.
chain_pair_pae_min *lowest predicted aligned error between two chains - do they touch at all?*	Lowest PAE between any part of chain i and chain j. Separates binders from non-binders better than ipTM does.
has_clash / fraction_disordered *Sanity checks.*	has_clash trips when over half a chain clashes, or a chain has 100+ clashing atoms. Disorder is measured by accessible surface area.

Two traps: pTM collapses below ~20 tokens, so our 13-residue SpyTag reads ≈0.05 and is still right. And large disordered regions drag pTM and ipTM down for the whole complex — check the PAE for the part you care about before binning anything.

To know more about alphafold3 scoring: https://www.ebi.ac.uk/training/online/courses/alphafold/alphafold-3-and-alphafold-server/alphafold-server-your-gateway-to-alphafold-3/interpreting-results-from-alphafold-server/

LIMITS

Five things AlphaFold 3 will not do

1	**It does not score binding**	A confident pose is not an affinity. Independent work found AF3 still places ATP and heme in their usual sites even when the binding residues are mutated away.
2	**One static structure — and it may be the wrong one**	No ensemble, no dynamics. E3 ligases come back closed even when you ask for the ligand-free state.
3	**It can hallucinate order**	Spurious structure in disordered regions, sometimes as a tidy helix rather than obvious spaghetti. pLDDT well below 50 is the marker.
4	**Chirality and clashes**	Roughly 4% chirality violations on the PoseBusters set; identical chains can be predicted overlapping. Check has_clash.
5	**It does not replace an experiment**	A confident structure is a hypothesis, not a result.

WHERE TO GO NEXT

Take this back to your own complex

MODEL PARAMETERS

github.com/google-deepmind/alphafold3

Request form, input and output docs

QUEST DOCUMENTATION

rcdsdocs.it.northwestern.edu/systems/quest/software/alphafold/alphafold-quest.html

Systems → Quest → Software → AlphaFold

WORKING SCRIPTS

github.com/nuitrcs/Protein-Modeling

alphafold/v3.0.0

Ask Us

quest-help@northwestern.edu

Consultations: bit.ly/rcdsconsult